Node.js Request Lifecycle for Merch4Change

This document serves as a technical architectural guide for the Merch4Change backend, detailing the runtime environment, startup procedures, and the execution path of incoming API requests.

1. The Core Idea: Non-Blocking Architecture

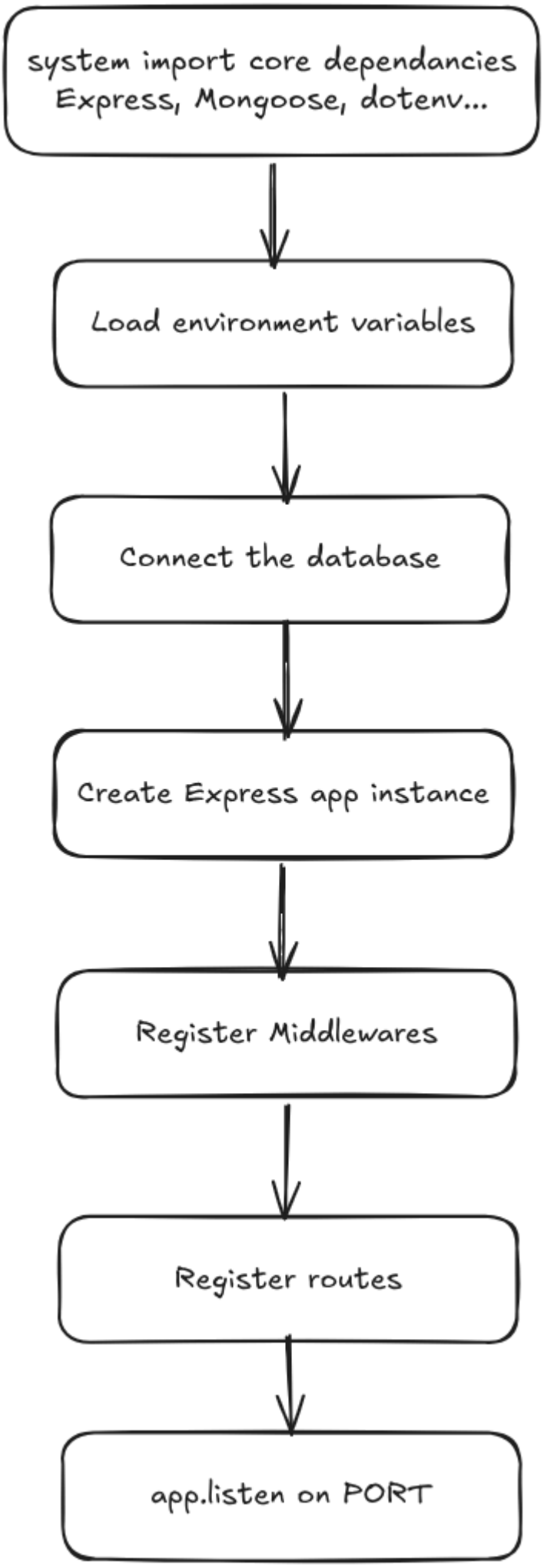
Node.js executes JavaScript on a single thread using an Event Loop. Unlike traditional servers that create a new thread for every incoming connection, the Merch4Change backend handles concurrent requests by delegating I/O operations (like database queries to MongoDB Atlas) to the system kernel or thread pool.

- Asynchronous Execution: When the server performs a database read or writes to a file, the main thread remains free to accept new requests while waiting for the operation to complete.
- The Waiter Analogy: The event loop acts like a single waiter in a restaurant. They take an order (request), hand it to the kitchen (I/O), and immediately move to the next table. Once the food is ready, they return to the original table to deliver the response.

2. Startup Flow: Booting the Merch4Change Server

The initialization process follows a strict sequence to ensure all dependencies and configurations are loaded before the server begins listening for traffic.

1. Module Resolution: The system imports core dependencies including Express, Mongoose, and dotenv.
2. Configuration Loading: Environment variables (e.g., PORT, MONGO_URI) are loaded into process.env.
3. Database Connection: The backend establishes an asynchronous connection to the MongoDB Atlas cluster.
4. Middleware Registration: Global utilities are registered to the Express instance, including:
    - cookie-parser: Essential for handling the HttpOnly JWT refresh token flow.
    - cors: Manages cross-origin resource sharing between the Vite-based frontend and Node.js backend.
    - express.json(): Parses incoming JSON payloads.
5. Route Definition: API routes (e.g., /api/auth, /api/profile, /api/marketplace) are mounted to specific path handlers.
6. Server Listen: The application executes app.listen(PORT), notifying the OS to begin forwarding connections to the process.


system import core dependancies
Express, Mongoose, dotenv...
Load environment variables
Connect the database
Create Express app instance
Register Middlewares
Register routes
app.listen on PORT

3. Request Lifecycle: Path of an API Call

When a client initiates a request (e.g., a POST to /api/auth/login), it traverses several layers before a response is returned.

Phase 1: Ingress and Parsing

The request enters the Express middleware stack. The order of these operations is critical for security and data availability.

- CORS Check: Validates that the request origin is allowed.
- Body Parsing: express.json() transforms the raw request stream into a usable req.body object.
- Cookie Extraction: cookie-parser populates req.cookies, allowing access to the JWT refresh token stored in the browser's HttpOnly cookie.

Phase 2: Security and Validation

Before reaching the controller, the request must pass through specific security gates.

- Authentication Middleware: Custom logic (e.g., verifyToken) checks for a valid JWT in the Authorization header.
- Request Validation: Middleware (utilizing validators in src/validators/) ensures the payload contains properly formatted data.

Phase 3: Controller Execution and Async I/O

The matched route handler (Controller) takes over to perform business logic.

- Database Interaction: The controller calls Mongoose methods (e.g., User.findOne()).
- Non-Blocking Wait: Because DB calls are asynchronous, the event loop handles other tasks while MongoDB processes the query.

- Resume and Logic: Once the data returns, the controller may perform password comparison (via Bcrypt) or token generation (via Jsonwebtoken).

Phase 4: Egress and Response

The final stage involves preparing and sending the data back to the client.

- Response Formatting: The controller sends a JSON response using a standardized utility (e.g., apiResponse).
- Token Setting: For login/refresh actions, the backend sets the new refresh token in an HttpOnly cookie.
- Termination: The HTTP connection is closed, and the event loop continues to the next pending event.

Flow chart:
https://excalidraw.com/#room=895e5001bfe57fda271d,zcELYqgI1VPQuDTC5AZXQw

4. Key Backend Components

- **Component: Middlewares** | Responsibility: Global and route-specific request processing | Relevant Files: src/middlewares/auth.js, notFound.js
- **Component: Controllers** | Responsibility: Core business logic and DB coordination | Relevant Files: auth.controller.js, profile.controller.js
- **Component: Models** | Responsibility: Schema definitions for MongoDB | Relevant Files: User.js, Order.js, Product.js
- **Component: Validators** | Responsibility: Input data sanitization and schema checking | Relevant Files: auth.validator.js, profile.validator.js

References

This doc:

https://docs.google.com/document/d/1-fIyWaSJgPNYAuxys65GwLSmrj6qOO0GDHi6x1HnCWY/edit?usp=sharing

Author: Malith Sandanayake
Last Updated: 2026/07/19